

从 PLC 到 PC 控制系统的转型实战课程

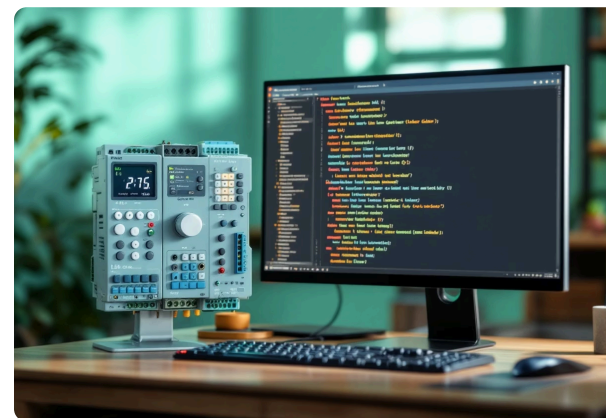
第5课：面向对象程序设计_2 (OOP: Object-Oriented Programming)

物件导向程式设计_2

讲师: [May] | 课程时长: 50 分钟

OOP-Based Control Programming

Instructor: [May] | Class Duration: 50 Minutes



从传统工业控制到现代软件控制的转型之旅

回顾上节课 (Review Last Class)

-  理解了 类 (Class) 与 对象 (Object)
-  学会定义属性 (Attribute) 与方法 (Method)
-  编写了 Dog 类、LED 类，实现基本实例化与调用
-  完成了一个简单的**三色灯控制器**：red / green / blue / show / exit

课程目标与概述

学习目标

- **理解封装 (Encapsulation)**: 隐藏对象内部状态, 仅暴露对外接口
 - *Understand how encapsulation protects internal object state and provides a clean interface.*
- **掌握组合与聚合 (Composition & Aggregation)**: 构建复杂对象系统
 - *Build complex systems by combining objects together.*
- **应用封装与组合, 构建三色灯控制系统 (Red / Green / Blue)**
 - *Apply encapsulation and composition to build a CLI-controlled RGB lamp system.*
- **为后续继承与多态的代码结构打基础**
 - *Lay the foundation for class hierarchies, abstraction, and polymorphism.*

什么是封装（Encapsulation）？

封装的意义：

- 将对象的属性与方法绑定在一起
- 隐藏内部实现细节，仅暴露“必要接口”
- 防止外部直接修改状态 → 提高代码安全性与可维护性

```
class LED:
    def init(self, name):
        self.__status = False # 私有属性

    def toggle(self):
        self.__status = not self.__status # 只能通过方法修改
```

Python 私有属性语法

```
class Lamp:
    def __init__(self, color: str):
        self.color = color    # 公有 Public
        self._status = False  # 受保护（约定）Protect
        self.__is_on = False  # 私有（Name Mangling）Private
```

💡 约定：单下划线“_”供子类/模块内部使用，双下划线禁止外部直接调用。

Python 私有属性语法

属性写法	类型	是否可在类外访问	用途和语义
self.color	公有属性	✅ 可以	默认公开接口，供外部正常访问和修改
self._status	受保护属性	✅ 可以（但约定不要）	“我希望你不要直接访问”——类外可以访问，但被认为是内部使用
self.__is_on	私有属性	❌ 不能直接访问	Python 会做 name mangling，真正变量名变成 <i>ClassName_is_on</i> ，防止外部访问

你送朋友一个 📁 礼盒 GiftBox，里面有一些礼品：

- 🍫 一盒巧克力 → 朋友可以**直接取出**
- ✉️ 一个信封（写着“私人”）→ 朋友**可以打开，但你不推荐**
- 🔒 一个上锁的小盒子 → 朋友**无法打开**，除非破解

```
class Lamp:
    def __init__(self, color):
        self.color = color    # 公有
        self._status = False  # 受保护
        self.__is_on = False  # 私有

lamp = Lamp("red")

print(lamp.color)    # OK
print(lamp._status)  # OK，但不推荐（受保护）
print(lamp.__is_on)  # 会报错：AttributeError

# 真正的私有变量名：
print(lamp._Lamp__is_on) # OK（Name Mangling 方式访问）
```

如何实现封装？

✓ 使用“私有属性”+“对外接口”

- `__name` 表示私有属性，无法被类外访问
- 提供 `get_status()` / `toggle()` 等方法来访问和修改

```
class LED:

    def __init__(self, name):
        self.name = name
        self.__status = False

    def toggle(self):
        self.__status = not self.__status

    def get_status(self):
        return self.__status
```

封装接口：方法设计

```
class Lamp:
    def toggle(self):
        self.__is_on = not self.__is_on
        return self.status()

    def status(self):
        return f'{self.color} 灯：{'开' if self.__is_on else '关'}'
```

- 公开方法（API） = 控制器按钮
- 私有数据 = 受保护的开关状态

✓ 任何更改均通过方法流转，内部细节可随时重构。

什么是组合（Composition）？

 把多个对象组合起来形成更复杂的系统

- 一个对象中包含其他对象作为属性
- 常用于模拟“部件组成整体”的结构关系

```
class LampSystem:  
    def __init__(self):  
        self.red = LED("红灯")  
        self.green = LED("绿灯")  
        self.blue = LED("蓝灯")
```

举例说明

组合（**Composition**）——“人体和心脏”

```
class Heart:
    def __init__(self):
        self.beats = 0

class Human:
    def __init__(self):
        self.heart = Heart() # 组合：Human 拥有自己的 heart

person = Human()
```

- Heart 是 Human 的一部分（part-of）
- 一旦 Human 被删除，Heart 也没了（生命绑定）

✅ 聚合（Aggregation）——“老师和学生”

```
class Student:
    def __init__(self, name):
        self.name = name

class Teacher:
    def __init__(self, students):
        self.students = students # 聚合：学生可被多个老师共享

s1 = Student("Alice")
s2 = Student("Bob")
t1 = Teacher([s1, s2])
t2 = Teacher([s1]) # s1 被多个老师使用
```

- 学生 s1、s2 独立于教师 t1、t2 存在
- 即使某个老师对象消失，学生对象仍然存在

✓ 一句话定义

关系	定义
组合 Composition	强关联：子对象完全依赖于父对象的生命周期，父对象销毁时，子对象也随之消失。
聚合 Aggregation	弱关联：子对象 独立存在 ，可以在多个父对象中共享，父对象销毁不影响子对象。

总结对比表：

特性	组合 (Composition)	聚合 (Aggregation)
关系	强拥有关系 (Part-of)	弱拥有关系 (Has-a)
生命周期	父对象销毁 → 子对象销毁	子对象独立存在
例子	人类 → 心脏、公司 → 部门	老师 → 学生、项目 → 成员
代码表现	子对象在 <code>init()</code> 中创建	子对象作为参数传入

何时用聚合？

- 你希望对象能在多个地方共享使用
- 子对象可能独立存在或复用，不属于“专属部分”
- 例如：
 - 学生属于多个班级
 - 同一个 Logger（日志）被多个组件复用

原则：

- 优先组合 → 更强封装
- 聚合适用于共享对象或依赖反转

示例：Lamp 类 (封装)

```
class Lamp:
    def __init__(self, color: str):
        self.color = color
        self.__is_on = False

    def toggle(self):
        self.__is_on = not self.__is_on

    def status(self):
        return f'{self.color} lamp is {'ON' if self.__is_on else 'OFF'}'
```

现在每个灯都可自我管理开关状态。

示例：LampGroup (组合)

```
class LampGroup:
    def __init__(self):
        self.red = Lamp("Red")
        self.green = Lamp("Green")
        self.blue = Lamp("Blue")

    def toggle(self, color: str):
        lamp = getattr(self, color, None)
        if lamp:
            lamp.toggle()

    def show(self):
        for lamp in (self.red, self.green, self.blue):
            print(lamp.status())
```

组合后，灯组拥有统一接口：toggle()、show()

示例：LampGroup (聚合)

```
# 先定义 Lamp 类（简单封装）
class Lamp:
    def __init__(self, name):
        self.name = name
        self._on = False

    def toggle(self):
        self._on = not self._on

    def status(self):
        return f"{self.name} 状态：{'开' if self._on else '关'}"

# 聚合关系：LampGroup 不拥有灯，而是“接收”它们
class LampGroup:
    def __init__(self, red_lamp, green_lamp, blue_lamp):
        self.lamps = {
            "red": red_lamp,
            "green": green_lamp,
            "blue": blue_lamp
        }

    def toggle(self, color: str):
        lamp = self.lamps.get(color)
        if lamp:
            lamp.toggle()
        else:
            print(" 没有这个颜色的灯！")

    def show(self):
        for lamp in self.lamps.values():
            print(lamp.status())

# 外部先创建灯实例
red  = Lamp("红灯")
green = Lamp("绿灯")
blue = Lamp("蓝灯")

# 灯传入 LampGroup，形成聚合关系
group = LampGroup(red, green, blue)

# 控制逻辑
group.toggle("red")
group.show()
```

聚合的优势

```
# 同一个 red 灯可以被多个系统共享
group1 = LampGroup(red, green, blue)
group2 = LampGroup(red, Lamp("新绿"), Lamp("新蓝"))

group1.toggle("red")
group2.show() # 红灯状态也受到了 group1 的影响！
```

小结 & 思考

1. 封装提高安全性与可维护性
2. 组合让系统模块化、易扩展

思考题：

- 如何让 `LampGroup` 支持任意数量的颜色灯？
- 组合与继承有哪些取舍？

课后作业：三色灯控制器

任务描述

- **目标：**实现命令行三色灯控制程序
- **要求：**
 - a. 定义 `Lamp` 类（属性：`color`，私有 `__is_on`）
 - b. 定义 `LampGroup` 类（组合 3 个 `Lamp` 对象）
 - c. 用户命令：`red/green/blue` 切换，`show` 显示，`exit` 退出
 - d. 代码需遵守封装原则，不得直接修改私有属性
- **进阶挑战**（选做）：
 - 支持任意颜色列表初始化 (`LampGroup(colors=[...])`)